ChatGPT

Save as PDF

Pasted text.txt
Document

this is the launch file for a vision system in my rosbot, help me understand what it does and uses

I'm reading through the launch file and I'll break down both the ROS 2 launch mechanics and the specific vision pipeline it sets up.

Thought for 14s

This launch file sets up a small ROS 2 vision pipeline for object recognition. At a high level, it takes camera images from your robot, republishes them in a way that matches the QoS requirements of the detector, and then runs `find_object_2d` on that republished image stream. Pasted text

The file starts by importing the ROS 2 launch tools it needs:

- `LaunchDescription` to define the whole launch file
- `DeclareLaunchArgument` so topics and paths can be overridden at runtime
- `SetEnvironmentVariable` and `LogInfo` for logging behavior
- `Node` to start ROS nodes
- `LaunchConfiguration` and `EnvironmentVariable` so arguments and env vars can be used dynamically. Pasted text

Inside `generate_launch_description()`, it defines the default topics:

- RGB image input: `/oak/rgb/image_raw/compressed`
- Repeated RGB topic: `/oak/rgb/image_raw/compressed/repeat`
- Depth topic: `/camera/depth/image_raw`
- Camera info topic: `/camera/depth/camera_info`
- `use_compressed = 'true'` meaning the image stream is expected to be compressed. Pasted text

That means this system is currently built around the OAK camera's compressed RGB feed, not the older commented-out `/camera/color/image_raw` options. The repeated topic is just the original topic with `'/repeat'` added to the end. Pasted text

The two `SetEnvironmentVariable(...)` calls change ROS logging so messages are printed straight to stdout and are not buffered. In practice, that usually makes logs appear

immediately in the terminal, which is useful when debugging launch and vision nodes.

Pasted text

The launch arguments make the file configurable without editing code. The important ones are:

- `gui` : whether `find_object_2d` opens its GUI, default `true`
- `image_topic` : the incoming camera topic
- `image_topic_repeat` : the republished reliable topic
- `use_compressed` : whether the repeater should treat the stream as compressed
- `objects_path` : where trained/reference object images are stored
- `settings_path` : config file for `find_object_2d`
- depth and camera-info topics, though those are not actively used right now.

Pasted text

`objects_path` is built from the environment variable `AIIL_CHECKOUT_DIR` , and the launch file logs that variable to the console. So this file expects your workspace path to be available in the environment, and then appends:
`/humble_workspace/src/par_coursework/objects_example`
as the folder containing object templates or training images for detection. Pasted text

The first active node is:

```
package='find_object_2d', executable='find_object_2d'
```

This is the actual object recognition node. It is configured with:

- `subscribe_depth: False`
- `gui` : controlled by the launch argument
- `objects_path` : where known objects are loaded from
- `settings_path` : the `.ini` config file. Pasted text

The most important remap is:

- `image → image_topic_repeat`

So `find_object_2d` is not subscribing directly to the camera topic. It listens to the repeated topic instead. The commented-out remappings for depth and camera info show that depth support was considered, but is currently disabled. Also, since `subscribe_depth` is set to `False` , this node is running in 2D image-only mode. Pasted text

The second active node is:

```
package='aiil_rosbot_demo', executable='best_effort_repeater'
```

This node subscribes to the original image topic and republishes it onto the `/repeat` topic. Its parameters are:

- `sub_topic_name` : original camera topic
- `repeat_topic_name` : repeated topic
- `use_compressed` : whether the message type/handling is compressed. Pasted text

This repeater exists because, as the comment says, `find_object_2d` only uses reliable QoS, while your incoming camera stream is probably best-effort QoS. That mismatch is a common ROS 2 problem: a reliable subscriber often will not receive messages from a best-effort publisher. So the repeater acts like a bridge:

1. subscribe to the camera's best-effort stream
2. republish in a form that `find_object_2d` can consume reliably. Pasted text

The depth repeater and camera-info repeater nodes are both commented out. That tells you the system may have been designed to support RGB + depth later, but right now only the RGB image repeater is active. So depth data is declared as launch arguments, but it is not actually part of the running pipeline yet. Pasted text

So the full data flow is basically:

OAK camera compressed RGB image
→ `best_effort_repeater`
→ `/oak/rgb/image_raw/compressed/repeat`
→ `find_object_2d`
→ object detection based on images stored in `objects_path` . Pasted text

What this launch file uses:

- ROS 2 launch system
- `find_object_2d` package for 2D object recognition
- `aiil_rosbot_demo` custom repeater node
- OAK RGB compressed camera stream
- an environment variable `AIIL_CHECKOUT_DIR`
- optionally a GUI and saved object templates. Pasted text

A few practical observations:

- It is not doing deep-learning inference directly in this file; it is launching `find_object_2d` , which is more of a feature/template-based object finder.
- It is not currently using depth, despite having depth-related arguments.
- The key purpose of the custom repeater is QoS compatibility.

- The object database lives in `objects_example`, so detection depends on what reference objects are stored there.

Pasted text

In plain English: this launch file starts an object finder for your robot camera, but because your camera publishes images with QoS settings the detector does not like, it inserts a repeater node in the middle to make the image stream usable. Then `find_object_2d` looks for known objects from a saved folder and can show a GUI while doing it.

Pasted text

I can also go through it line by line and explain exactly what each launch argument and node parameter does in ROS terms.

Sources

Sources

Thought for 9s

If `find_object_2d` is running **on the bot** and the camera topic still shows **0 publishers**, then the issue is now:

the camera driver is not publishing that topic on the bot

So `find_object_2d` is fine, but it has no image stream to read.

What this usually means

One of these is true:

- the camera node is not running
- the camera is publishing on a **different topic**
- the camera node crashed
- the camera is publishing only a compressed topic or another namespace
- you are checking from a terminal not sourced into the same ROS setup

Check this on the bot

Run these:

Bash

```bash
ros2 node list | grep camera
ros2 topic list | grep camera
ros2 topic list | grep image
ros2 topic info /camera/color/image_raw
ros2 topic info /camera/color/image_raw/compressed
```

What you want to find

You need to identify the topic that has:

```
Publisher count: 1
```

It may turn out the real published topic is something like:

- `/camera/color/image_raw/compressed`
- `/camera/image_raw`
- `/camera/image`
- another namespace entirely

Also check whether the camera node is actually alive

Earlier you had nodes like:

- `/camera/camera`
- `/camera/healthcheck_astra`

If those are missing now, the camera driver is not running.

So also run:

Bash

```bash
ros2 node list
```

and look for the camera node.

Most likely next fix

If only compressed has a publisher

Then either:

- remap `find_object_2d` to the actual published image topic setup you have working
- or get the raw image topic enabled from the camera driver

If no camera topics have publishers at all

Then the camera itself is not publishing, so you need to start/fix the camera node first.

Best next command sequence

Run exactly these on the bot:

Bash

```bash
ros2 node list | grep camera
ros2 topic list | grep -E 'camera|image'
ros2 topic info /camera/color/image_raw
ros2 topic info /camera/color/image_raw/compressed
```

Paste that output and I can tell you exactly which topic `find_object_2d` should subscribe to, or whether the camera node itself needs fixing.

there are no other camera topics

Thought for a few seconds

Then the issue is simpler now:

the camera is not publishing at all on the bot.

If there are no other camera/image topics, `find_object_2d` cannot work yet because it has nothing to subscribe to.

What this means

Right now you have:

- `find_object_2d` running
- no camera publishers
- no image stream

So the next thing to fix is **the camera driver / camera launch**, not the hazard code.

What to check next

Run these on the bot:

```bash
ros2 node list
```

Look for a camera node, something like:

- `/camera/camera`
- `/camera/...`
- `astra`
- `realsense`
- `orbbec`

Then run:

Bash

```bash
ros2 topic list
```

and confirm there is really nothing like:

- `/camera/color/image_raw`
- `/camera/color/image_raw/compressed`
- `/camera/image_raw`
- `/image_raw`

Most likely situation

One of these is probably true:

1. the camera node is not launched
2. the camera hardware is not detected
3. the camera driver crashed
4. you are on the wrong machine/session for the camera bring-up

Best next move

You need to start the camera separately first.

Since your robot looked like it was using an Astra/Orbbec-style camera earlier, the camera launch may be part of your robot bring-up, not part of your package.

So try to find how the camera is normally started on the bot.

Check for installed camera packages

Run:

Bash

```bash
ros2 pkg list | grep -E 'astra|orbbec|realsense|camera'
```

That will show what camera driver packages are available.

Check whether the camera node exists

Run:

Bash

```bash
ros2 node list | grep -i camera
```

If nothing relevant appears, the camera simply is not running.

Practical debugging order

Do this in order:

1

Check for camera packages:

```bash
ros2 pkg list | grep -E 'astra|orbbec|realsense|camera'
```

2

Check for camera nodes:

```bash
ros2 node list | grep -i camera
```

3

If no camera node is running, start the camera bring-up you normally use for the robot.

4

After launching it, verify:

```bash
ros2 topic list | grep camera
ros2 topic info /camera/color/image_raw
```

You need `Publisher count: 1` or more.

Important point

Do **not** keep debugging `find_object_2d` until the camera stream exists.

The correct dependency order is:

1. camera publishes images
2. `find_object_2d` subscribes to those images
3. detections appear on `/objectsStamped`
4. `hazard_manager` subscribes to detections

Right now you are still stuck at step 1.

Paste the output of:

Bash

```bash
ros2 pkg list | grep -E 'astra|orbbec|realsense|camera'
ros2 node list | grep -i camera
```

and I'll help you identify which camera launch to start.

/camera/healthcheck_astra

Thought for a few seconds

That tells you the camera **healthcheck** node is running, but **not the actual camera driver node**.

So the situation is:

- `/camera/healthcheck_astra` exists
- no image topics are being published
- therefore the Astra camera driver itself is not up

`healthcheck_astra` just monitors/checks the camera. It does **not** publish the image stream that `find_object_2d` needs.

What this means

Your next job is to start the **actual Astra camera node** or the normal robot bring-up that includes it.

Check what camera package is installed

Run:

Bash

```bash
ros2 pkg list | grep -E 'astra|orbbec'
```

You're looking for something like:

- `astra_camera`
- `orbbec_camera`
- similar camera driver packages

Check whether the camera device is visible

Run:

```bash
ls /dev/video*
lsusb
```

If the camera is connected properly, you should usually see:

- one or more `/dev/video*` devices
- a USB entry for the camera in `lsusb`

Most likely next step

You probably need to launch the robot's camera driver separately. Depending on what's installed, it may be something like:

```bash
ros2 launch astra_camera astra_camera.launch.py
```

or similar, but the exact command depends on the package you have installed.

Best next commands

Run these and paste the output:

```bash
ros2 pkg list | grep -E 'astra|orbbec'
ls /dev/video*
lsusb
```

That will tell us:

- which driver package exists
- whether the camera is actually detected by Linux
- what launch command to try next